

Microsoft Azure Cloud Data Processing Pipeline Implementation

Project Author: Esmeralda C. Cabrera Ventura

Program: PGP in Cloud Computing

Platform: Microsoft Azure

Azure Cloud Data Processing Pipeline Implementation

1. Project Overview

This project implements an end-to-end cloud data processing workflow using Microsoft Azure services. The system integrates compute, storage, and database components to process invoice data using a Python program executed on a virtual machine.

The workflow begins with a text-based invoice file that is processed by a Python application running on an Ubuntu Virtual Machine in Azure. The script converts the invoice data into CSV format, uploads the CSV file to Azure Blob Storage, and inserts the structured data into an Azure Database for MySQL instance.

The project demonstrates practical cloud engineering skills including infrastructure deployment, remote system administration, Python automation, cloud storage integration, and database configuration.

2. Project Purpose

The purpose of this project is to demonstrate the practical implementation of an end-to-end cloud workflow using Azure services. The project highlights how cloud resources can be combined to create a functional data processing pipeline.

Specifically, the project demonstrates the ability to:

- Deploy Azure cloud infrastructure
- Configure and access a Linux virtual machine
- Execute a custom Python data processing application

- Upload files to Azure Blob Storage
- Insert structured data into a managed MySQL database
- Validate the successful execution of cloud workloads

This project reinforces real-world cloud engineering practices including environment configuration, secure remote access, service integration, and validation of deployed cloud resources.

3. Project Objectives

The objectives of this project were to:

1. Deploy a resource group to organize all Azure resources.
2. Create and configure an Azure Storage Account.
3. Deploy an Ubuntu Virtual Machine with SSH access enabled.
4. Create an Azure Database for MySQL Flexible Server instance.
5. Modify the provided Python program with the correct credentials.
6. Transfer application files from the local machine to the virtual machine.
7. Install required dependencies on the VM.
8. Create the MySQL database environment.
9. Execute the Python application to process invoice data.
10. Validate the creation of the CSV file in Blob Storage.
11. Verify successful insertion of data into the MySQL database.

3. System Architecture

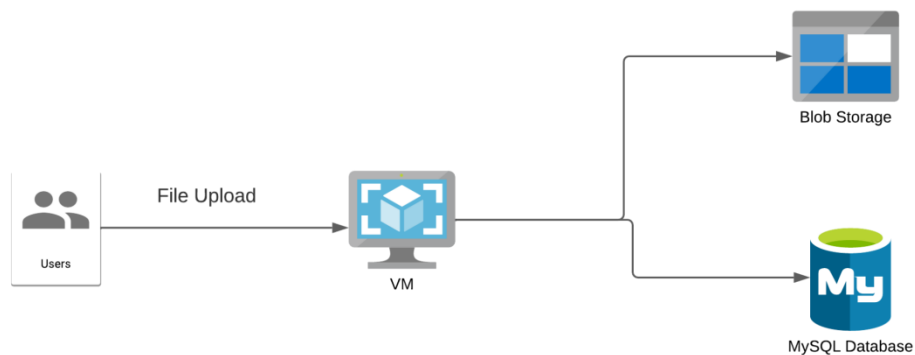
The architecture of the implemented system integrates three primary Azure services:

- Azure Virtual Machine (Compute Layer)
- Azure Blob Storage (Storage Layer)
- Azure Database for MySQL Flexible Server (Database Layer)

The Python application running on the virtual machine performs the following operations:

1. Reads invoice data from a text file
2. Converts the data to CSV format
3. Uploads the CSV file to Azure Blob Storage
4. Inserts the invoice records into a MySQL database

Figure 1 – System Architecture Diagram



4. Technologies Used

Cloud Platform	Microsoft Azure
Compute	Azure Virtual Machine (Ubuntu)
Storage	Azure Blob Storage
Database	Azure Database for MySQL
Programming Language	Python
Data Processing	Pandas
Connectivity	Azure Storage SDK / MySQL Connector
OS	Ubuntu Linux

5. Step-by-Step Implementation

Step 1 – Create Azure Infrastructure Resources

The first phase of the project involved provisioning the necessary cloud infrastructure resources in Microsoft Azure.

The required infrastructure consisted of:

- Azure Resource Group
- Azure Storage Account
- Ubuntu Virtual Machine
- Azure Database for MySQL Flexible Server

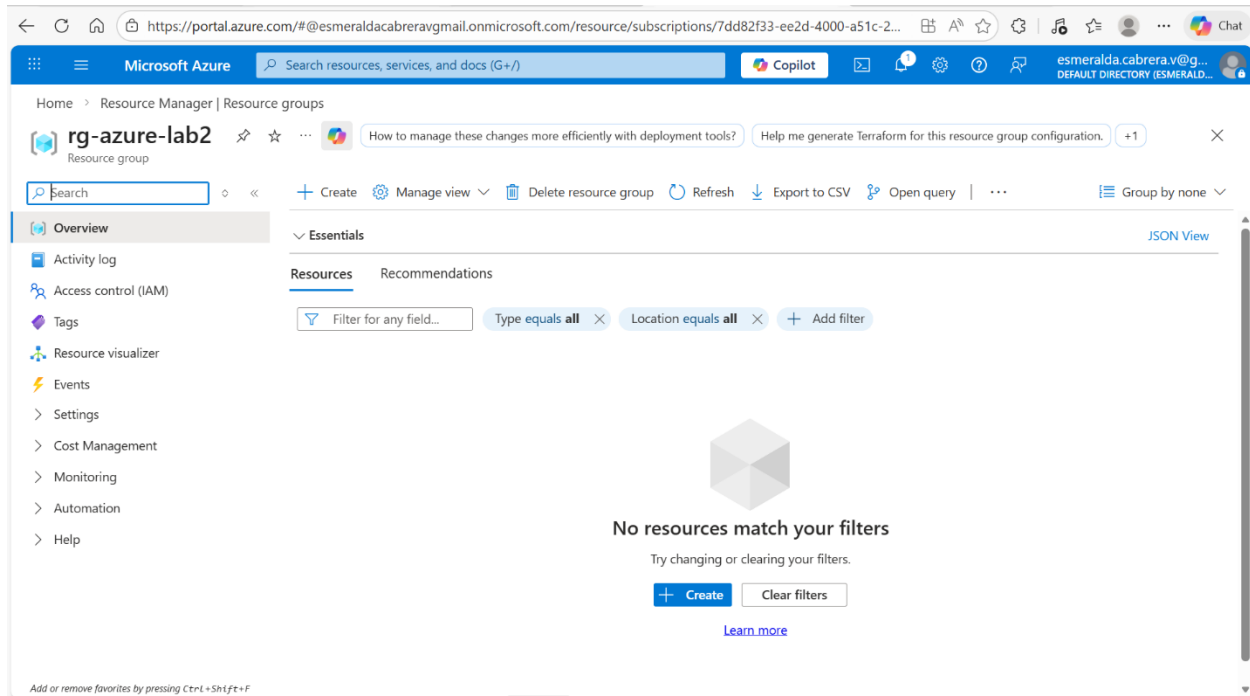
Each resource was deployed through the Azure Portal.

Resource Group Creation

A Resource Group was created to logically organize all resources associated with the project.

Resource groups allow Azure administrators to manage multiple resources as a single unit.

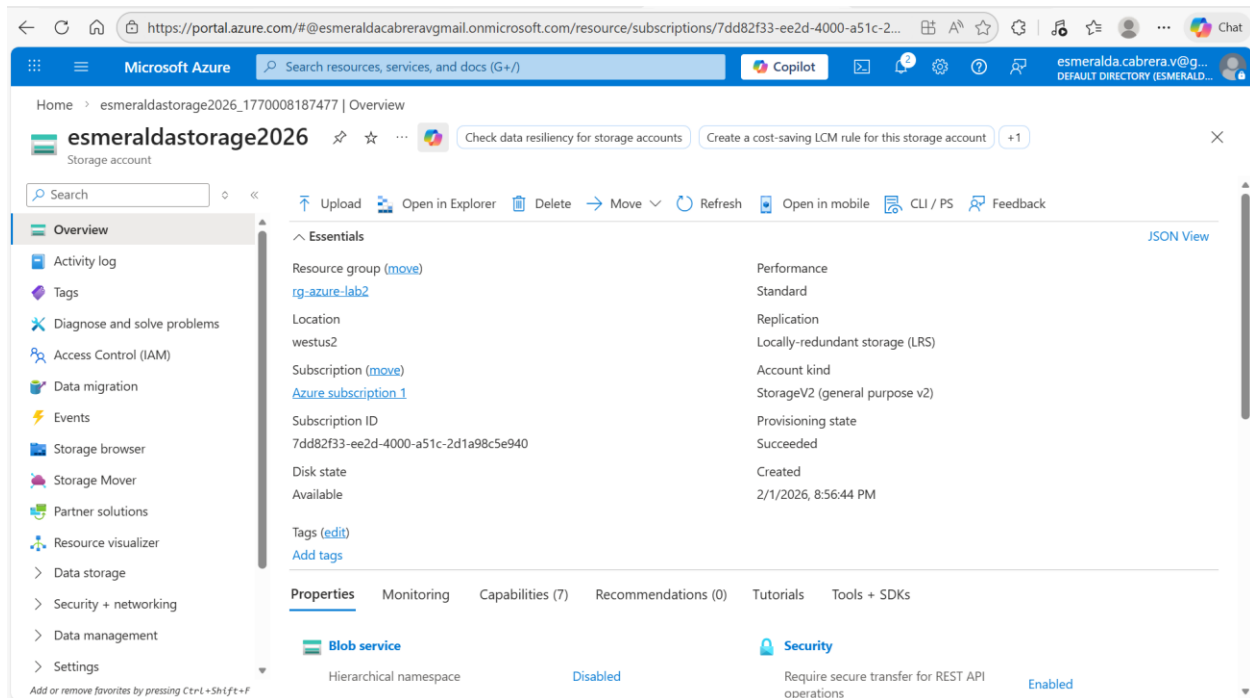
Figure 2 – Resource Group Creation



An Azure Storage Account was created to host the Blob Storage container used to store the CSV file generated by the Python program.

After the storage account was deployed, the access keys section was opened and the connection string associated with Key 1 was copied. This connection string was later used within the Python script to authenticate and upload files to Blob Storage.

Figure 3 – Storage Account Creation



Virtual Machine Creation

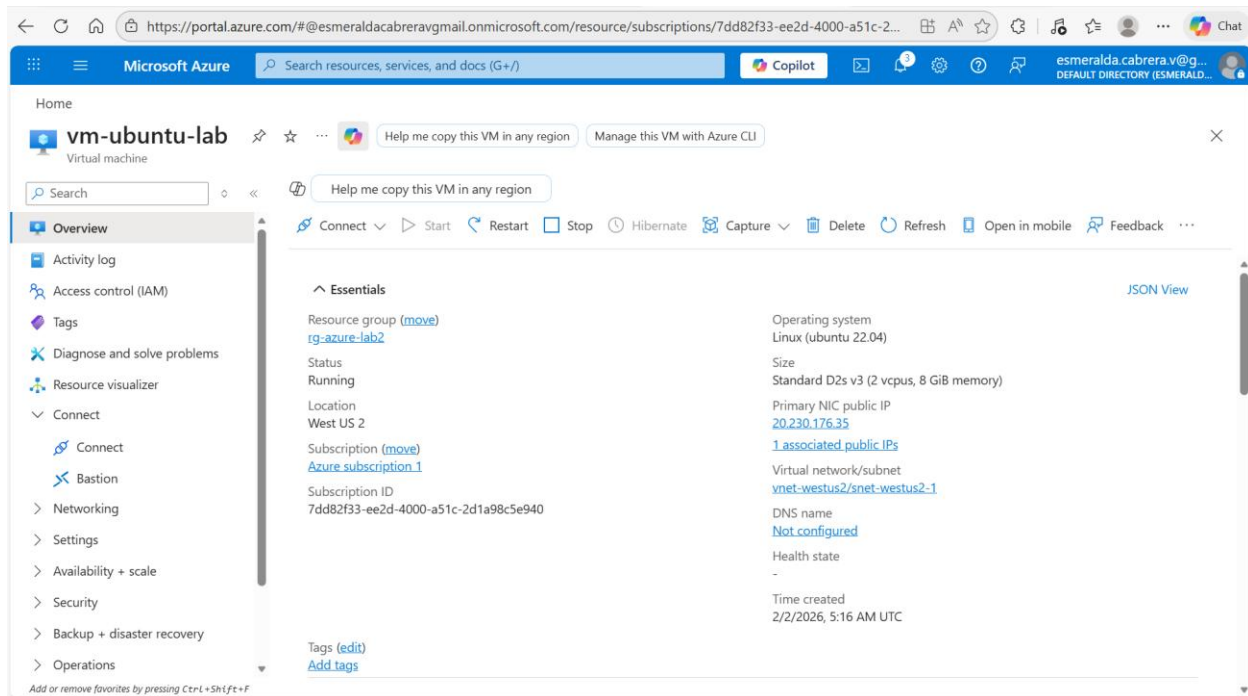
An Ubuntu virtual machine was deployed to serve as the compute environment for executing the Python data processing application.

Key configuration settings included:

- Operating System: Ubuntu 20.04
- Authentication Type: SSH Public Key
- Inbound Port: SSH (Port 22)

The virtual machine acts as the execution environment where the Python program processes invoice data and interacts with other Azure services.

Figure 4 – Virtual Machine Creation



MySQL Database Server Creation

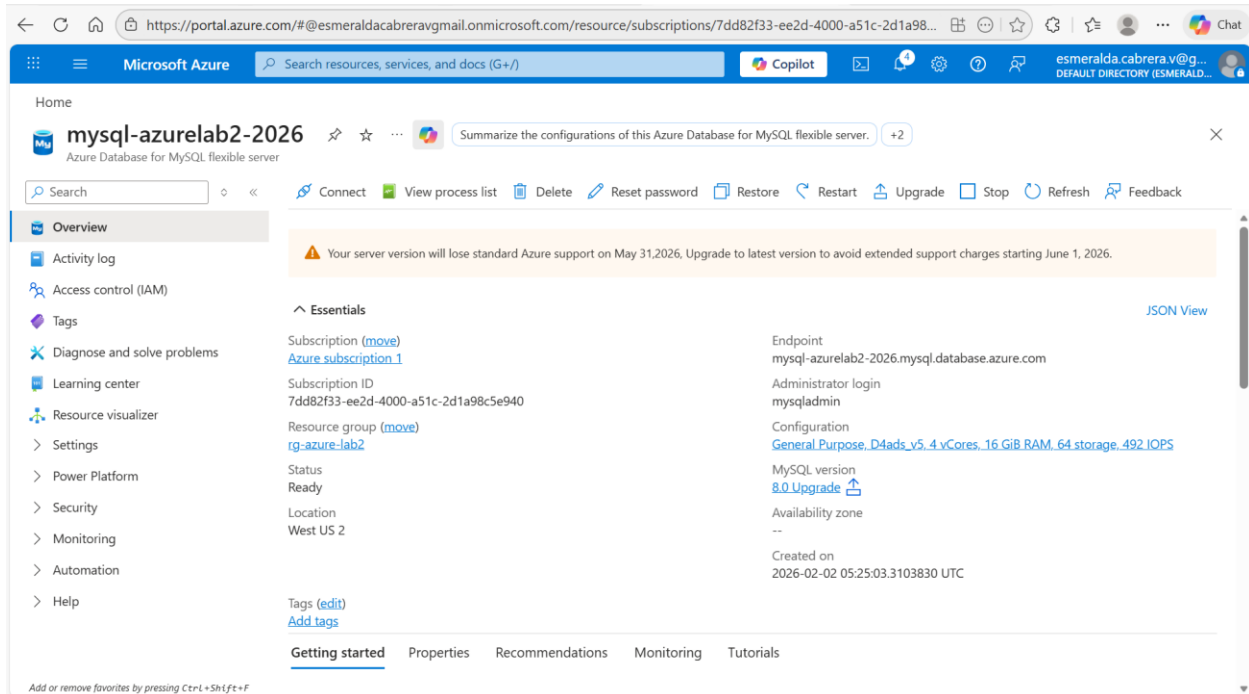
An Azure Database for MySQL Flexible Server instance was created to host the relational database used to store the processed invoice data.

During the configuration process, the following parameters were defined:

- Server name
- Administrator username
- Administrator password
- Public access enabled
- Azure services allowed access

Once the server was created, the fully qualified server name was recorded for use in the Python script configuration.

Figure 5 – MySQL Database Server Creation



Step 2 – Prepare and Configure the Python Program

The provided Python script (process.py) was modified to include the appropriate configuration values required to connect to Azure services.

The following values were updated in the script:

- Azure Storage Account connection string
- MySQL database server name
- MySQL database username
- MySQL database password

These changes enabled the Python program to authenticate and interact with the storage account and the database server.

Script Used to Setup Environment

```
sudo apt update
```

```
sudo apt install python3
```

```
sudo apt install python3-pip
```

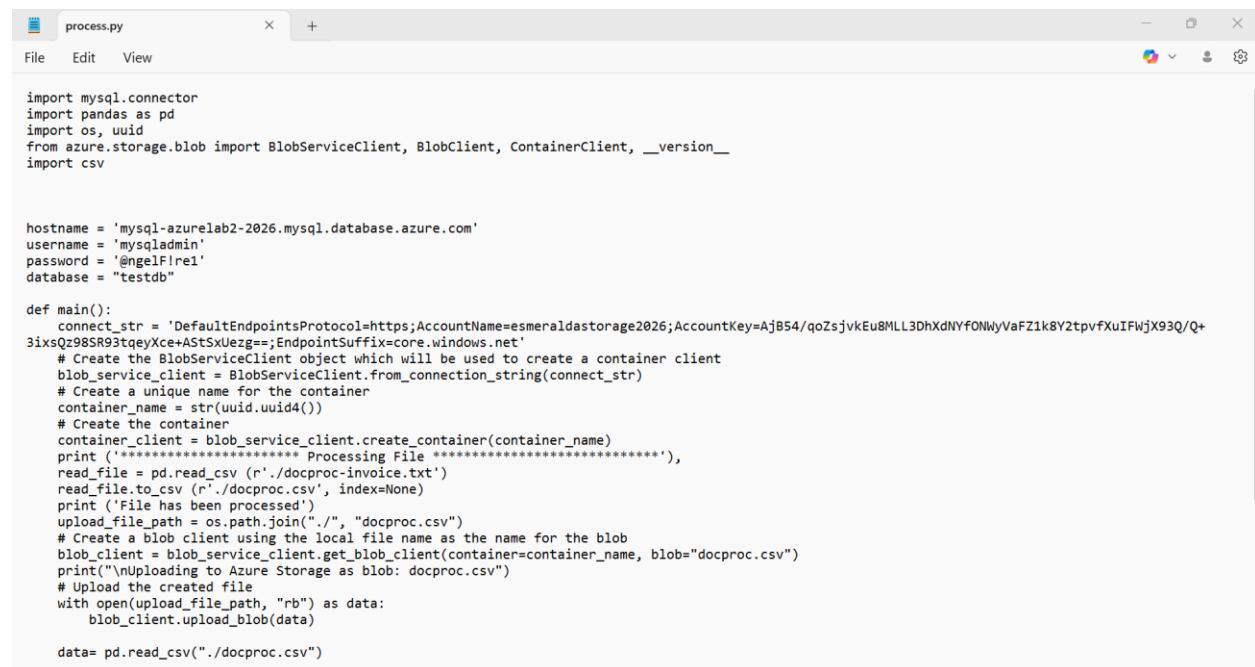
```
sudo pip3 install pandas
```

```
sudo pip3 install azure-storage-blob
```

```
sudo pip3 install mysql-connector-python
```

```
sudo apt install mysql-client-core-8.0
```

Figure 6 – Updated Python Script Configuration



```
process.py
File Edit View
import mysql.connector
import pandas as pd
import os, uuid
from azure.storage.blob import BlobServiceClient, BlobClient, ContainerClient, __version__
import csv

hostname = 'mysql-azurelab2-2026.mysql.database.azure.com'
username = 'mysqladmin'
password = '@ngelF!re1'
database = "testdb"

def main():
    connect_str = 'DefaultEndpointsProtocol=https;AccountName=esmeraldastorage2026;AccountKey=Aj854/qoZsjvkEu8MLL3DhXdNYfONWYVaFZ1k8Y2tpvfXuIFWjX93Q/Q+3ixsQz98SR93tqeyXce+AStSXUezg==;EndpointSuffix=core.windows.net'
    # Create the BlobServiceClient object which will be used to create a container client
    blob_service_client = BlobServiceClient.from_connection_string(connect_str)
    # Create a unique name for the container
    container_name = str(uuid.uuid4())
    # Create the container
    container_client = blob_service_client.create_container(container_name)
    print('***** Processing File *****'),
    read_file = pd.read_csv(r'./docproc-invoice.txt')
    read_file.to_csv(r'./docproc.csv', index=None)
    print('File has been processed')
    upload_file_path = os.path.join("./", "docproc.csv")
    # Create a blob client using the local file name as the name for the blob
    blob_client = blob_service_client.get_blob_client(container=container_name, blob="docproc.csv")
    print("\nUploading to Azure Storage as blob: docproc.csv")
    # Upload the created file
    with open(upload_file_path, "rb") as data:
        blob_client.upload_blob(data)

data= pd.read_csv("./docproc.csv")
```

Step 3 – Transfer Files to the Virtual Machine

The Python script and the invoice text file were transferred from the local system to the Ubuntu virtual machine using Secure Copy Protocol (SCP).

Example command used:

```
scp -i <keyfile.pem> process.py ubuntu@<VM-IP>:/home/ubuntu
```

The SCP command securely copies files between systems using SSH encryption.

Figure 7 – Copying Files Using SCP

```
ubuntu@vm-ubuntu-lab: ~  
-a----- 11/3/2025 11:16 PM 59 costs.csv  
-a----- 11/29/2025 11:27 PM 258 CSC670-65107-2512_report_Esmeralda_Cabrera_Ventura_71894_2025-11-29_2327_PST.csv  
-a----- 11/27/2024 2:16 AM 123933 foodhub_order (1).csv  
-a----- 12/8/2024 2:23 PM 123933 foodhub_order (2).csv  
-a----- 11/25/2024 9:52 AM 123933 foodhub_order.csv  
-a----- 6/30/2025 8:43 PM 1033 High-Protein_Meal_Plan.csv  
-a----- 2/16/2025 6:30 PM 208964 marketing_campaign.csv  
-a----- 6/3/2024 9:38 AM 169 payment-history.csv  
-a----- 11/23/2024 1:42 AM 112852 StockData.csv  
-a----- 11/29/2024 1:47 PM 848 Summary_Insights_from_Dataset.csv  
-a----- 11/29/2024 3:10 PM 1487 Summary_of_Dataset_Insights.csv  
  
PS C:\Users\Esmeralda Cabrera> scp -i "C:\Users\Esmeralda Cabrera\Downloads\vm-ubuntu-lab_key.pem" "  
>> "C:\Users\Esmeralda Cabrera\Downloads\azurelab2\script.py"  
>> ubuntu@20.230.176.35:/home/ubuntu/  
C:\WINDOWS\System32\OpenSSH\scp.exe: stat local "C:/Users/Esm  
PS C:\Users\Esmeralda Cabrera> dir "C:\Users\Esmeralda Cabrera\Downloads\process*","C:\Users\Esm  
ice*"  
  
Directory: C:\Users\Esmeralda Cabrera\Downloads  
  
Mode LastWriteTime Length Name  
----  
-a----- 2/1/2026 9:39 PM 2492 process.py  
-a----- 2/1/2026 9:33 PM 251 docproc-invoice.txt  
  
PS C:\Users\Esmeralda Cabrera> scp -i "C:\Users\Esmeralda Cabrera\Downloads\vm-ubuntu-lab_key.pem" "C:\Users\Esm  
ads\process.py" ubuntu@20.230.176.35:/home/ubuntu/  
process.py 100% 2492 51.8KB/s 00:00  
PS C:\Users\Esmeralda Cabrera> scp -i "C:\Users\Esmeralda Cabrera\Downloads\vm-ubuntu-lab_key.pem" "C:\Users\Esm  
ads\docproc-invoice.txt" ubuntu@20.230.176.35:/home/ubuntu/  
docproc-invoice.txt 100% 251 4.6KB/s 00:00  
PS C:\Users\Esmeralda Cabrera> ls -l
```

Step 4 – Access the Virtual Machine

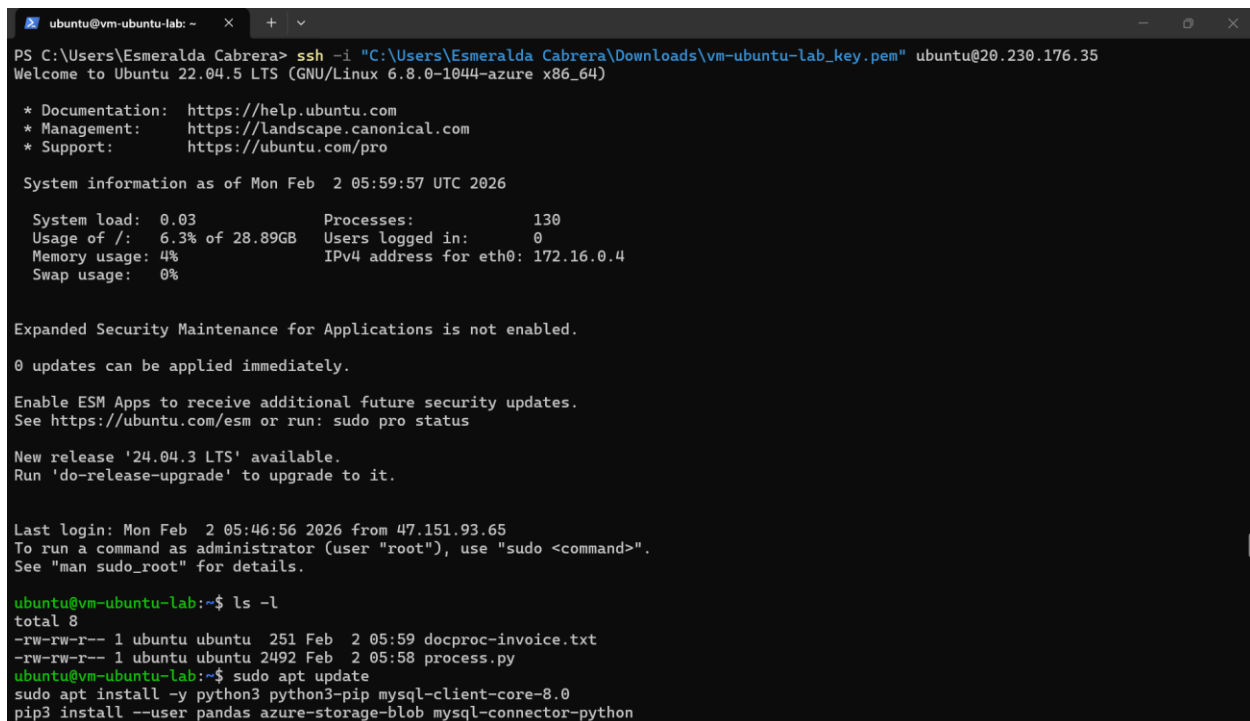
After transferring the files, the Ubuntu virtual machine was accessed remotely using SSH.

Example command:

```
ssh -i <keyfile.pem> ubuntu@<VM-IP>
```

SSH provides secure remote administration capabilities, allowing administrators to execute commands, install software, and manage system configuration.

Figure 8 – SSH Login to Ubuntu Virtual Machine



```
PS C:\Users\Esmeralda Cabrera> ssh -i "C:\Users\Esmeralda Cabrera\Downloads\vm-ubuntu-lab_key.pem" ubuntu@20.230.176.35
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1044-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Feb  2 05:59:57 UTC 2026

System load:  0.03               Processes:            130
Usage of /:   6.3% of 28.89GB    Users logged in:     0
Memory usage: 4%                IPv4 address for eth0: 172.16.0.4
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Mon Feb  2 05:46:56 2026 from 47.151.93.65
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@vm-ubuntu-lab:~$ ls -l
total 8
-rw-rw-r-- 1 ubuntu ubuntu 251 Feb  2 05:59 docproc-invoice.txt
-rw-rw-r-- 1 ubuntu ubuntu 2492 Feb  2 05:58 process.py
ubuntu@vm-ubuntu-lab:~$ sudo apt update
sudo apt install -y python3 python3-pip mysql-client-core-8.0
pip3 install --user pandas azure-storage-blob mysql-connector-python
```

Step 5 – Install Required Software Dependencies

Once connected to the virtual machine, several required packages were installed to prepare the environment for executing the Python program.

Installed components included:

- Python3
- pip3

- pandas library
- Azure Blob Storage SDK
- MySQL connector for Python
- MySQL client tools

These packages allow the Python application to process data and interact with both Azure storage and the MySQL database.

Figure 9 – Software Installation Completed

```

ubuntu@vm-ubuntu-lab: ~
No VM guests are running outdated hypervisor (qemu) binaries on this host.
Collecting pandas
  Downloading pandas-2.3.3-cp310-cp310-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (12.8 MB)
    12.8/12.8 MB 48.5 MB/s eta 0:00:00
Collecting azure-storage-blob
  Downloading azure_storage_blob-12.8.0-py3-none-any.whl (431 kB)
    431.5/431.5 KB 37.4 MB/s eta 0:00:00
Collecting mysql-connector-python
  Downloading mysql_connector_python-9.6.0-cp310-cp310-manylinux_2_28_x86_64.whl (34.5 MB)
    34.5/34.5 MB 40.2 MB/s eta 0:00:00
Requirement already satisfied: pytz>=2020.1 in /usr/lib/python3/dist-packages (from pandas) (2022.1)
Collecting tzdata>=2022.7
  Downloading tzdata-2025.3-py2.py3-none-any.whl (348 kB)
    348.5/348.5 KB 36.7 MB/s eta 0:00:00
Collecting numpy>=1.22.4
  Downloading numpy-2.2.6-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (16.8 MB)
    16.8/16.8 MB 72.6 MB/s eta 0:00:00
Collecting python-dateutil>=2.8.2
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
    229.9/229.9 KB 20.0 MB/s eta 0:00:00
Collecting azure-core>=1.30.0
  Downloading azure_core-1.38.0-py3-none-any.whl (217 kB)
    217.8/217.8 KB 23.8 MB/s eta 0:00:00
Collecting typing-extensions>=4.6.0
  Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
    44.6/44.6 KB 5.3 MB/s eta 0:00:00
Requirement already satisfied: cryptography>=2.1.4 in /usr/lib/python3/dist-packages (from azure-storage-blob) (3.4.8)
Collecting isodate>=0.6.1
  Downloading isodate-0.7.2-py3-none-any.whl (22 kB)
Requirement already satisfied: requests>=2.21.0 in /usr/lib/python3/dist-packages (from azure-core>=1.30.0->azure-storage-blob) (2.25.1)
Requirement already satisfied: six>=1.5 in /usr/lib/python3/dist-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)
Installing collected packages: tzdata, typing-extensions, python-dateutil, numpy, mysql-connector-python, isodate, pandas, azure-core, azure-storage-blob
WARNING: The scripts f2py and numpy-config are installed in '/home/ubuntu/.local/bin' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed azure-core-1.38.0 azure-storage-blob-12.8.0 isodate-0.7.2 mysql-connector-python-9.6.0 numpy-2.2.6 pandas-2.3.3 python-dateutil-2.9.0.post0 typing-extensions-4.15.0 tzdata-2025.3
ubuntu@vm-ubuntu-lab:~$ python3 --version
python3 --version
pip3 --version
mysql --version

```

Step 6 – Configure the Database

The Azure MySQL server was accessed from the virtual machine using the MySQL command-line client.

Example command:

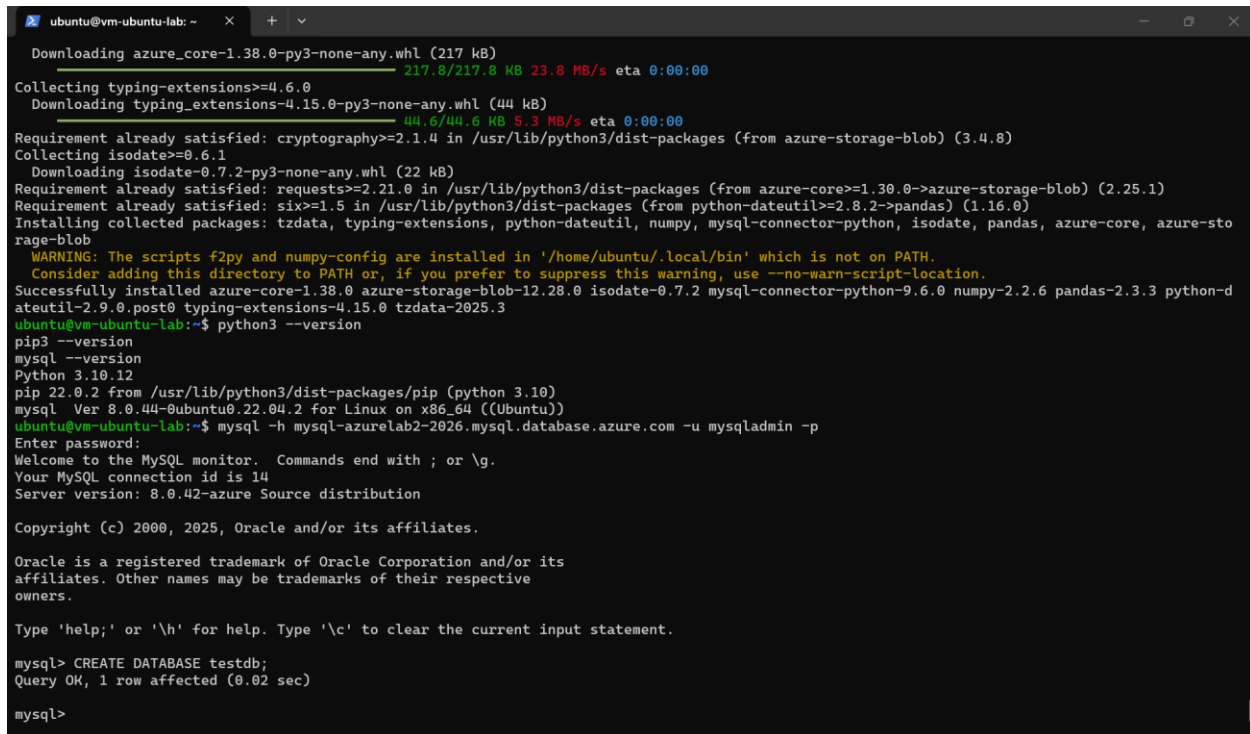
```
mysql -h <server-name> -u <username> -p
```

After authentication, a new database was created using the SQL command:

```
CREATE DATABASE testdb;
```

This database was used to store the processed invoice records generated by the Python application.

Figure 10 – Database Configuration



```
ubuntu@vm-ubuntu-lab: ~  
Downloading azure_core-1.38.0-py3-none-any.whl (217 kB)  
217.8/217.8 KB 23.8 MB/s eta 0:00:00  
Collecting typing_extensions>=4.6.0  
Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)  
44.6/44.6 KB 5.3 MB/s eta 0:00:00  
Requirement already satisfied: cryptography>=2.1.4 in /usr/lib/python3/dist-packages (from azure-storage-blob) (3.4.8)  
Collecting isodate>=0.6.1  
Downloading isodate-0.7.2-py3-none-any.whl (22 kB)  
Requirement already satisfied: requests>=2.21.0 in /usr/lib/python3/dist-packages (from azure-core>=1.30.0->azure-storage-blob) (2.25.1)  
Requirement already satisfied: six>=1.5 in /usr/lib/python3/dist-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)  
Installing collected packages: tzdata, typing_extensions, python-dateutil, numpy, mysql-connector-python, isodate, pandas, azure-core, azure-storage-blob  
WARNING: The scripts f2py and numpy-config are installed in '/home/ubuntu/.local/bin' which is not on PATH.  
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.  
Successfully installed azure-core-1.38.0 azure-storage-blob-12.28.0 isodate-0.7.2 mysql-connector-python-9.6.0 numpy-2.2.6 pandas-2.3.3 python-dateutil-2.9.0.post0 typing_extensions-4.15.0 tzdata-2025.3  
ubuntu@vm-ubuntu-lab:~$ python3 --version  
python3 --version  
mysql --version  
Python 3.10.12  
pip 22.0.2 from /usr/lib/python3/dist-packages/pip (python 3.10)  
mysql Ver 8.0.44-0ubuntu0.22.04.2 for Linux on x86_64 ((Ubuntu))  
ubuntu@vm-ubuntu-lab:~$ mysql -h mysql-azurelab2-2026.mysql.database.azure.com -u mysqladmin -p  
Enter password:  
Welcome to the MySQL monitor. Commands end with ; or \g.  
Your MySQL connection id is 14  
Server version: 8.0.42-azure Source distribution  
  
Copyright (c) 2000, 2025, Oracle and/or its affiliates.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
mysql> CREATE DATABASE testdb;  
Query OK, 1 row affected (0.02 sec)  
  
mysql>
```

Step 7 – Execute the Python Application

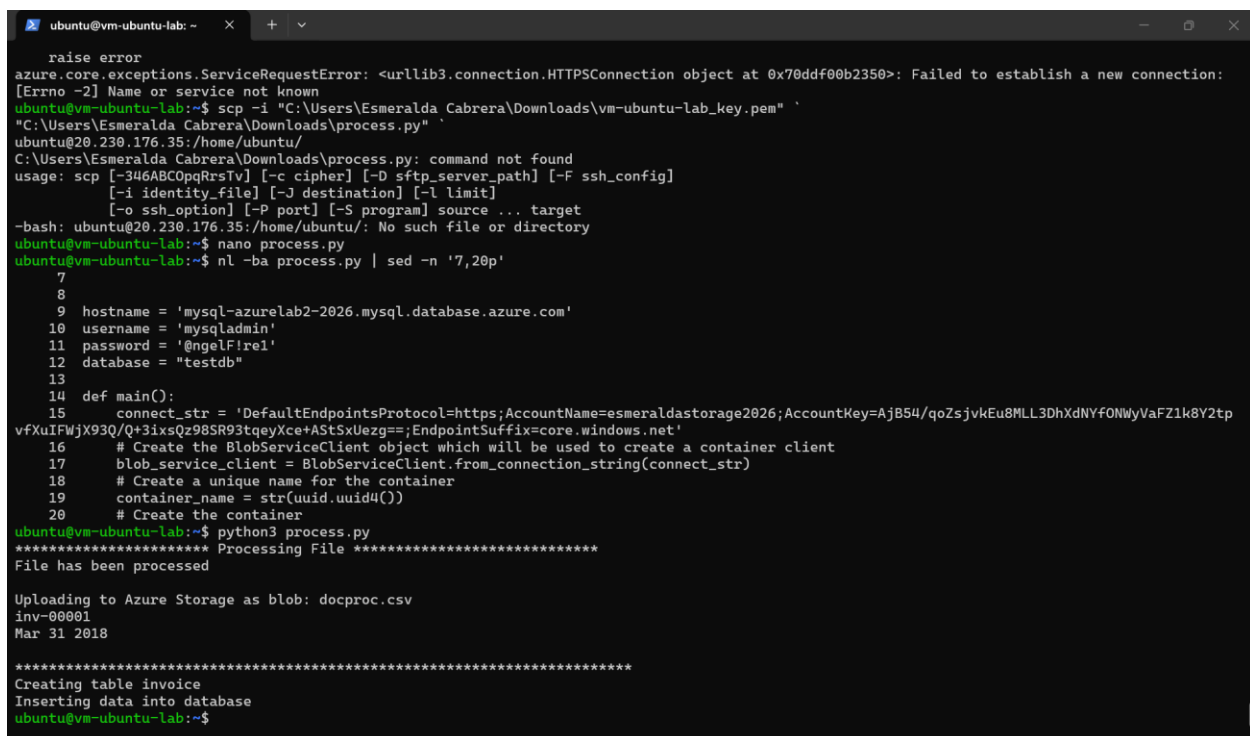
The Python program was executed on the virtual machine using the following command:

```
python3 process.py
```

The program performed the following operations:

1. Read the invoice text file
2. Converted the data into CSV format
3. Uploaded the CSV file to Azure Blob Storage
4. Inserted the invoice records into the MySQL database

Figure 11 – Running the Python Program



```
raise error
azure.core.exceptions.ServiceRequestError: <urllib3.connection.HTTPSConnection object at 0x70ddf00b2350>: Failed to establish a new connection:
[Errno -2] Name or service not known
ubuntu@vm-ubuntu-lab:~$ scp -i "C:\Users\Esmeralda Cabrera\Downloads\vm-ubuntu-lab_key.pem" '
"C:\Users\Esmeralda Cabrera\Downloads\process.py"
ubuntu@20.230.176.35:/home/ubuntu/
C:\Users\Esmeralda Cabrera\Downloads\process.py: command not found
usage: scp [-346ABCOpqRrsTv] [-c cipher] [-D sftp_server_path] [-F ssh_config]
[-i identity_file] [-J destination] [-l limit]
[-o ssh_option] [-P port] [-S program] source ... target
-bash: ubuntu@20.230.176.35:/home/ubuntu/: No such file or directory
ubuntu@vm-ubuntu-lab:~$ nano process.py
ubuntu@vm-ubuntu-lab:~$ nl -ba process.py | sed -n '7,20p'
 7
 8
 9 hostname = 'mysql-azurelab2-2026.mysql.database.azure.com'
10 username = 'mysqladmin'
11 password = '@ngelF!rel'
12 database = "testdb"
13
14 def main():
15     connect_str = 'DefaultEndpointsProtocol=https;AccountName=esmeraldastorage2026;AccountKey=AjB54/qoZsjvkEu8MLL3DhXdNYfONWyVaFZ1k8Y2tp
vfXuIFWjX93Q/Q+3ixsQz98SR93tqeyXce+ASt$XUezg==;EndpointSuffix=core.windows.net'
16     # Create the BlobServiceClient object which will be used to create a container client
17     blob_service_client = BlobServiceClient.from_connection_string(connect_str)
18     # Create a unique name for the container
19     container_name = str(uuid.uuid4())
20     # Create the container
ubuntu@vm-ubuntu-lab:~$ python3 process.py
***** Processing File *****
File has been processed

Uploading to Azure Storage as blob: docproc.csv
inv-00001
Mar 31 2018

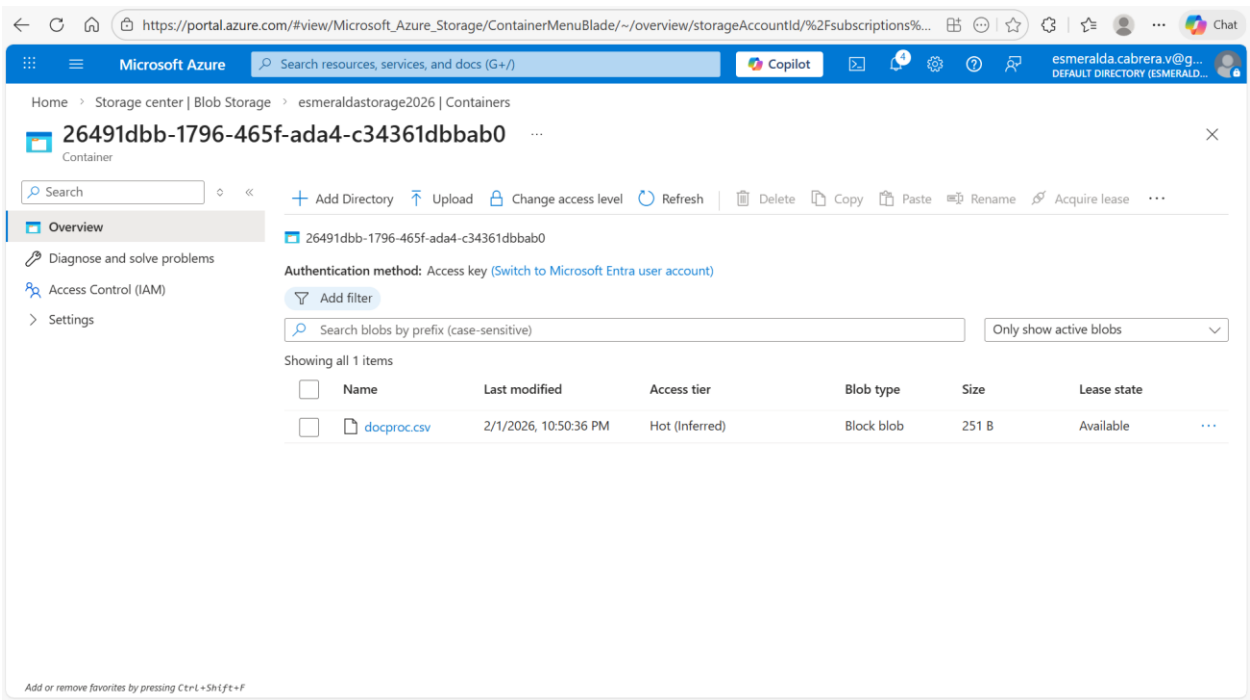
*****
Creating table invoice
Inserting data into database
ubuntu@vm-ubuntu-lab:~$
```

Step 8 – Verify Blob Storage Output

After the Python script completed execution, the Azure Storage Account was inspected to confirm that the CSV file had been successfully uploaded.

The Blob Storage container displayed the generated CSV file created by the Python program.

Figure 12 – Blob Storage Container Showing Generated CSV File



Step 9 – Verify Database Data Entry

The final validation step involved confirming that the invoice records were successfully inserted into the MySQL database.

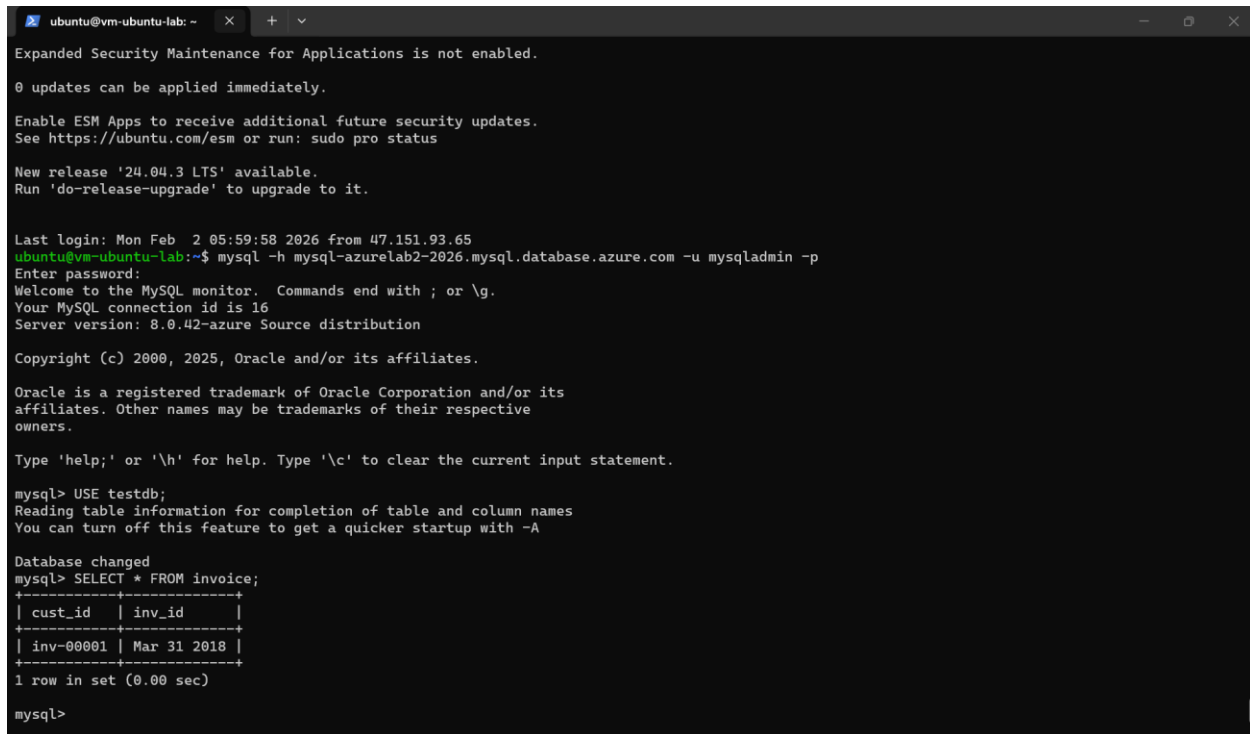
Using the MySQL client, the following commands were executed:

USE testdb;

SELECT * FROM invoice;

The query results confirmed that the invoice data was successfully stored in the database table.

Figure 13 – Database Data Entry Verification

A terminal window titled 'ubuntu@vm-ubuntu-lab: ~' showing the process of connecting to a MySQL database and verifying data. The terminal output includes system updates, login information, and a successful SQL query execution. The query 'SELECT * FROM invoice;' returns a single row with columns 'cust_id' and 'inv_id'.

```
ubuntu@vm-ubuntu-lab: ~  
Expanded Security Maintenance for Applications is not enabled.  
0 updates can be applied immediately.  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
New release '24.04.3 LTS' available.  
Run 'do-release-upgrade' to upgrade to it.  
Last login: Mon Feb  2 05:59:58 2026 from 47.151.93.65  
ubuntu@vm-ubuntu-lab:~$ mysql -h mysql-azurelab2-2026.mysql.database.azure.com -u mysqladmin -p  
Enter password:  
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 16  
Server version: 8.0.42-azure Source distribution  
Copyright (c) 2000, 2025, Oracle and/or its affiliates.  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
mysql> USE testdb;  
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup with -A  
Database changed  
mysql> SELECT * FROM invoice;  
+-----+-----+  
| cust_id | inv_id |  
+-----+-----+  
| inv-00001 | Mar 31 2018 |  
+-----+-----+  
1 row in set (0.00 sec)  
mysql>
```

12. Technical Skills Demonstrated

This project demonstrated hands-on experience with the following cloud engineering skills:

- Azure infrastructure provisioning
- Resource group management
- Virtual machine deployment and configuration
- Secure remote access using SSH
- File transfer using SCP
- Python automation in cloud environments
- Azure Blob Storage integration

- MySQL database configuration
- Cloud-based data processing pipelines

6. Conclusion

This project successfully implemented a cloud-based data processing pipeline using multiple Azure services. The integration of compute, storage, and database components demonstrated how cloud infrastructure can be used to automate data processing workflows.

The Ubuntu virtual machine hosted a Python application that processed invoice data, generated a CSV file, uploaded it to Azure Blob Storage, and inserted structured records into an Azure Database for MySQL instance. Verification steps confirmed the successful upload of the CSV file and the correct insertion of data into the database.

The project provided valuable practical experience with Azure cloud services, Linux system administration, Python automation, and relational database management in a cloud environment.